

电子设计实践基础实验报告

一、实验目的

- 掌握 MCU 任务调度机制：**通过实现多任务轮转调度，理解时间片管理与中断响应机制。
- 实现 LCD 自定义字符显示：**学习 LCD1602 的 CGRAM 自定义字符功能，完成中文显示。
- 无源蜂鸣器音频控制：**通过 PWM 技术生成方波信号，模拟音乐播放。
- 综合应用定时器、中断与外设通信：**结合键盘扫描、TWI 通信等技术，完成完整系统设计。

二、实验环境与工具

- 硬件：**ATMEGA8A 开发板、LCD1602 模块、4×4 矩阵键盘、无源蜂鸣器、杜邦线。
- 软件：**AVR-GCC 编译器、WinAVR 库、TWI 通信协议、AtmelStudio。
- 核心代码：**基于 avr/io.h 与 avr/interrupt.h 的 C 语言编程。

三、实验内容与实现

1. MCU 简单任务调度（OS 模拟）

设计思路

通过结构体数组管理多个任务，利用定时器中断实现时间片轮转。每个任务包含函数指针与执行间隔（单位：**ms**），主循环检查任务时间片是否耗尽，触发执行。

关键代码分析

```
typedef struct {  
    void (*ftask)(void); // 任务函数指针  
    int ticks;           // 任务执行间隔（时间片）  
} OSTASK;  
  
OSTASK runtask[MAX_TASKS] = {  
    {tp_task, 3},        // 任务 1：统计触摸开关次数（每 3ms 执行）  
    {Rgb_show, 7},       // 任务 2：RGB 灯状态更新（每 7ms 执行）  
    {lcd_show, 19}       // 任务 3：LCD 显示更新（每 19ms 执行）  
};
```

- **任务结构体**：OSTASK 包含任务函数指针与时间片计数器。
- **主循环调度**：遍历任务数组，若 ticks 为 0 则执行任务并重置时间片。
- **中断服务程序（ISR）**：定时器溢出时，所有任务的 ticks 减 1，实现时间片递减。

其中的重点问题：我们分析会发现在给出的事例当中已经有了 3 个部分了，具体如下所示

1. **tp_task()** 任务：触摸计数任务

功能：检测触摸开关的状态变化，并在每次按下触摸开关时增加计数。开关连接在 PB0 引脚上，任务会检测触摸开关从低电平到高电平的变化，每次触摸都会使 tp_cnt 计数器加 1。

执行周期：每次检测到触摸开关状态的变化时触发该任务。

2. **Rgb_show()** 任务：RGB LED 显示任务

功能：根据触摸计数 tp_cnt 的值来控制 RGB LED 的显示状态。

RGB LED 连接到 PC0, PC1, PC2 引脚。任务根据触摸计数的不同值来改变 LED 的状态，例如：在触摸计数达到特定值时开启或关闭不同颜色的 LED。

执行周期：定期检查 tp_cnt 的值，并更新 LED 的显示状态。

3. **lcd_show()** 任务：LCD 显示任务

功能：将触摸计数 tp_cnt 的值显示在 LCD 屏幕上。任务会将计数值转换为字符串并更新显示内容，显示格式为“times: XX”，其中 XX 为当前的触摸计数。

执行周期：定期更新 LCD 屏幕的显示内容，确保实时显示触摸计数。

这三个任务是通过一个简单的调度器以定时的方式轮流执行的，确保每个任务都能适当的时间周期内得到处理。

这里面通过一个简单的 OSTASK 包含任务函数指针与时间片计数器的方式可以对此 3 个任务进行调度从而完成任务。这里需要对 ISR 进行些许调整才能完成，加入 TIMSK 和 TCCR0 对定时器处理后方可正常执行任务。

```
ISR(TIMER0_OVF_vect) {
    TCNT0 = 131;           // 初始值, 1ms后溢出
    TIMSK = (1<<TOIE0);    // 开启定时器0溢出中断
    TCCR0 = (1<<CS01);      // 8分频, 启动定时器
    for (uint8_t i=0; i<MAX_TASKS; i++) {
        if (runtask[i].ticks > 0) runtask[i].ticks--; // 时间片递减
    }
}
```

思考与优化

- 动态任务添加：通过链表或动态数组支持运行时任务增减。
- 问题：长时间任务可能导致其他任务延迟，需合理设置时间片。

2. LCD1602 自定义字符显示

实现步骤

1. 生成汉字字模：使用 5×8 点阵工具生成汉字（如“学号”）的二进制数据。
2. 写入 **CGRAM**：通过 LCD_Write()将自定义字符数据写入 CGRAM 地址（0x40~0x47）。
3. 显示字符：在 DDRAM 中调用自定义字符地址，实现中文显示。

关键代码分析

```

unsigned char sf_data[8][8] = {
    // 自定义字符“学”的 8×8 点阵数据
    {0x04, 0x04, 0x1F, 0x15, 0x15, 0x1F, 0x04, 0x04},
    // 其他字符数据...
};

void LCD_Write_CGR() {
    for (int i = 0; i < 8; i++) { // 8个自定义字符
        for (int j = 0; j < 8; j++) {
            LCD_Write(0x40 + i * 8 + j, 0); // 设置 CGRAM 地址
            LCD_Write_8Bit(sf_data[i][j], 1); // 写入点阵数据
        }
    }
}
}

```

- **CGRAM 地址计算：**每个字符占用 8 字节，地址从 0x40 开始连续分配。
- **显示自定义字符：**在 DDRAM 中写入字符地址（如 0x00 对应第一个自定义字符）。

因为要使用 LCD，所以首先添加头文件  twi_lcd.h  twi_fun.h，在这里面我们可以使用在课程群里发送的 LCD 字符显示的 exe 程序将我们自己的姓名进行拆解，非常便利准确无误，可以避免人工书写我们自己的名字的一些错误，将我自己的名字的杜晓阳拆为木，土，日，尧，卩，日，其显示的效果显著。为了使我们的学号显示在第一行，我们要将 `LCD_Write_String(0,0,"PB23051023");` 写入替代。同时利用左移的操作，可以使我们的学号和姓名显示在合适的位置上。

```

unsigned char sf_data[8][8]={ //自定义字符:最多8个, 可以是汉字
    {0x04,0x04,0x1F,0x0E,0x15,0x04,0x00,0x00}, //木
    {0x04,0x04,0x0E,0x04,0x04,0x1F,0x00,0x00}, //土
    {0x1E,0x12,0x12,0x1E,0x12,0x12,0x1E,0x00}, //日
    {0x08,0x1E,0x04,0x03,0x1F,0x0C,0x13,0x00}, //尧
    {0x18,0x14,0x18,0x14,0x18,0x10,0x00,0x00}, //包耳旁
    {0x1E,0x12,0x12,0x1E,0x12,0x12,0x1E,0x00}, //日
    {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00}, //0
    {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00} }; //0
void LCD_CGR_Write() //将自定义的8个字符数据写入CGRAM

```

问题与解决

- 字符闪烁: 初始显示时需清屏 (0x01 指令), 避免残留字符干扰。
- 编码匹配: 确保自定义字符在 DDRAM 中的地址与显示位置一致。

3. 无源蜂鸣器 PWM 控制

驱动原理

无源蜂鸣器需频率可调的方波信号驱动。通过调整 PWM 的占空比与频率, 控制音调与音量。

关键代码分析

```

#define F_CPU 8000000UL
unsigned int freq = 1000; // 初始频率 1kHz
unsigned char duty = 50; // 占空比 50%

ISR(INT0_vect) { // 外部中断 0: 调整占空比
    duty = (duty == 90) ? 10 : duty + 10;
}

ISR(INT1_vect) { // 外部中断 1: 调整频率
    freq = (freq > 2000) ? 30 : freq + 100;
}

```

```

}

int main(void) {
    // 配置 PB1 为输出 (OC1A)
    DDRB |= (1<<DDRB1);
    TCCR1A |= (1<<COM1A1) | (1<<WGM10); // 快速 PWM 模式
    TCCR1B |= (1<<CS11); // 8 分频
    OCR1A = (F_CPU / (2 * freq)) - 1; // 计算占空比 50%的初始值

    while (1) {
        // 动态调整占空比与频率
        OCR1A = (F_CPU / (2 * freq)) * duty / 100;
    }
}

```

- **PWM 生成：**使用 TCCR1A 配置快速 PWM 模式，OCR1A 决定高电平持续时间。
- **频率与占空比计算：**
 - 频率公式： $\text{freq} = F_CPU / (2 * OCR1A)$
 - 占空比公式： $\text{duty} = (OCR1A / (F_CPU / (2 * \text{freq}))) * 100\%$
- **中断调整：**通过外部中断（INT0/INT1）动态修改 freq 与 duty。

在此实验中要首先要将其 MCU 的频率调整为 8MHz，在此按照上一次实验操作即可，注意代码加入一些 `#define F_CPU 8000000UL`, `#include <avr/io.h>`, `#include <util/delay.h>`, `#include <avr/interrupt.h>`, 这里面我们写一个播放函数 `void play_note(uint16_t frequency, uint16_t duration_ms)`, 我们可以通过调用函数实现播放，同时这里面我们定义 2 个数组，一个用来存放音调，另外一个用来存放各个音调的播放时间，我们最后通过查谱可以得知一首歌的歌词，然后翻译即可。如下：

```

// 定义音符频率 (单位Hz, c调标准音高)
#define NOTE_C4 262 // 低音Do
#define NOTE_D4 294 // Re
#define NOTE_E4 330 // Mi
#define NOTE_F4 349 // Fa
#define NOTE_G4 392 // So
#define NOTE_A4 440 // La
#define NOTE_B4 494 // Si
#define NOTE_C5 523 // 中音Do
#define NOTE_G3 196 // 网页4中低音SO对应392Hz, 此处G3需降八度
// 两只老虎主旋律 (c调简谱对应频率)
const uint16_t melody[] = {
    NOTE_C4, NOTE_D4, NOTE_E4, NOTE_C4, // 两只老虎, 两只老虎
    NOTE_C4, NOTE_D4, NOTE_E4, NOTE_C4, // 跑得快, 跑得快
    NOTE_E4, NOTE_F4, NOTE_G4, // 一只没有耳朵
    NOTE_E4, NOTE_F4, NOTE_G4, // 一只没有尾巴
    NOTE_G4, NOTE_A4, NOTE_G4, NOTE_F4, NOTE_E4, NOTE_C4, // 真奇怪! 真奇怪!
    NOTE_G4, NOTE_A4, NOTE_G4, NOTE_F4, NOTE_E4, NOTE_C4, // (重复段)
    NOTE_C4, NOTE_G3, NOTE_C4 // 结尾音
};

// 每个音符的持续时间 (单位ms)
const uint16_t duration[] = {
    500, 500, 500, 500, // 四分音符 x4
    500, 500, 500, 500, // 四分音符 x4
    750, 250, 1000, // 附点四分+八分+二分
    750, 250, 1000, // 同上
    250, 250, 250, 250, 500, 500, // 八分音符群+四分
    250, 250, 250, 250, 500, 500, // 同上
    1000, 500, 1500, // 二分+四分+附点二分
    1000, 500, 1500 // 二分+四分+附点二分
};

```

通过函数的书写和调用完成播放即可

```

void play_note(uint16_t frequency, uint16_t duration_ms) {
    ICR1 = F_CPU / 8 / frequency; // 设置频率
    OCR1A = ICR1 / 2; // 设置占空比50%
    // 简化的延时
    while (duration_ms--) {
        _delay_ms(1); // 每次延迟3ms
    }
    OCR1A = 0; // 停止播放音符
}

// 播放歌曲《一闪一闪亮晶晶》
for (int i = 0; i < sizeof(melody) / sizeof(melody[0]); i++) {
    play_note(melody[i], duration[i]); // 播放每个音符
}

```

这里巧妙的运用循环完成计时的操作。

思考与改进

- **多音阶播放：**预定义音符频率表，通过定时器中断切换频率，实现乐谱播放。
 - **音量控制：**调整占空比可模拟音量变化，但需注意频率稳定性。
 - **问题：**高频下定时器溢出可能导致频率误差，需优化 OCR1A 计算。
-

四、结论

结论

- 通过任务调度实现了多任务并发管理，提升了 MCU 利用率。
- LCD 自定义字符功能解决了中文显示问题，但需注意 CGRAM 资源限制。
- 无源蜂鸣器的 PWM 控制验证了频率与占空比对音频的影响。

本实验围绕 MCU 多任务调度、中断处理及外设控制展开，成功实现了任务轮转调度、矩阵键盘输入、LCD 自定义字符显示及无源蜂鸣器音频控制。通过结构体管理任务队列，结合定时器中断实现时间片轮转，有效提升了 MCU 利用率；采用中断方式处理矩阵键盘，避免按键抖动与重复输入，动态更新按键状态；利用 LCD 的 CGRAM 自定义字符功能，生成汉字字模并显示学号与姓名；通过 PWM 技术控制蜂鸣器方波信号，实现频率与占空比可调的音频输出。实验综合运用了定时器、中断、TWI

通信等技术，验证了任务调度机制与外设协同工作的稳定性，为嵌入式系统设计奠定了实践基础。